

**SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**  
**Department of Computer Science and Engineering**  
**CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment**

---

|                |                                                                                                                                                 |               |                                                    |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------|---------------|----------------------------------------------------|
| Course Code:   | CSA10 / CSA1063                                                                                                                                 | Course Title: | Software Engineering                               |
| CO Assessed:   | CO2                                                                                                                                             | Assessment:   | Assessment Tool 2 – Architecture Design Assignment |
| Weightage:     | 20%                                                                                                                                             | Total Marks:  | 25                                                 |
| CO2 Statement: | <i>Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills</i> |               |                                                    |
| Student Name:  | Shaik Mohammad Sohan                                                                                                                            | Reg. No.:     | 192525042                                          |
| Date:          |                                                                                                                                                 | Duration:     | 60 minutes                                         |

**Code of Conduct**

*I Shaik Mohammad Sohan/192525042(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Shaik Mohammad Sohan

---

**Annexure A – Architecture Design Assignment**

---

**Instructions to Students:**

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

**Question**

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

**1. Analyze System Requirements**

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

**2. Select a Suitable Software Architecture**

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

**3. Draw the Software Architecture Diagram**

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

**4. Explain Components and Their Interactions**

- Describe the purpose and responsibilities of each architectural component.

- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

## 5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

## 6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

### Rubrics for Calculation

| Evaluation Criteria                              | Excellent (4 Marks)                                                                                                                                                                | Good (3 Marks)                                                          | Satisfactory (2 Marks)                                                  | Needs Improvement (1 Mark)                                        |
|--------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|-------------------------------------------------------------------------|-------------------------------------------------------------------|
| <b>System Requirement Analysis</b>               | Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture. | Analyzes most requirements with minor omissions or limited explanation. | Identifies only basic requirements with partial analysis.               | Requirement analysis is incomplete, inaccurate, or lacks clarity. |
| <b>Selection of Software Architecture</b>        | Selects the most appropriate architectural style with strong technical justification aligned to system requirements.                                                               | Selects a suitable architecture with reasonable justification.          | Architecture is partially suitable with limited justification.          | Inappropriate architecture selected or justification is missing.  |
| <b>Architecture Diagram and Component Design</b> | Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.                               | Diagram is mostly complete with minor omissions or notation issues.     | Diagram includes only essential components and lacks sufficient detail. | Diagram is incomplete, unclear, or technically incorrect.         |

|                                                               |                                                                                                                                                                         |                                                                            |                                                                                                        |                                                                                   |
|---------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| <b>Component Interaction and Quality Attribute Analysis</b>   | Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability. | Explains most components and quality attributes with minor gaps.           | Provides limited explanation of interactions and addresses only some quality attributes.               | Component interactions and quality attributes are poorly explained or missing.    |
| <b>Architectural Justification and Technical Presentation</b> | Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.            | Provides adequate justification with minor gaps; report is well organized. | Limited justification with minimal discussion of trade-offs or future scope; report needs improvement. | Justification is weak or absent; report lacks organization and technical clarity. |

| Criteria                                               | Mark      |
|--------------------------------------------------------|-----------|
| System Requirement Analysis                            | /5        |
| Selection of Software Architecture                     | /5        |
| Architecture Diagram and Component Design              | /5        |
| Component Interaction and Quality Attribute Analysis   | /5        |
| Architectural Justification and Technical Presentation | /5        |
| <b>TOTAL</b>                                           | <b>/5</b> |

# **Architecture Design Report**

## **Case Study 35 (SDG 12: Sustainable Food Waste Reduction Platform)**

### **CHAPTER 1-ANALYZE SYSTEM REQUIREMENTS**

#### **1.1 Problem Overview and System Objectives**

Restaurants, supermarkets, and event organizers regularly generate surplus edible food that is discarded because there is no fast, reliable way to connect it with nearby charities that serve vulnerable communities. The proposed platform — the Food Redistribution Platform (FRP) — addresses this gap by digitally matching food donors with verified charities in real time, aligning directly with SDG 12 (Responsible Consumption and Production).

##### **Core objectives:**

- Enable donors to quickly list surplus food (type, quantity, expiry window, pickup location)
- Enable registered charities to discover and claim available donations near them in real time
- Automate matching and notification so time-sensitive donations aren't missed
- Track the pickup/delivery lifecycle of every donation for accountability and safety
- Provide administrators with tools to verify organizations and monitor platform-wide impact
- Generate analytics on food saved, meals provided, and CO<sub>2</sub> emissions avoided

#### **1.2 Functional Requirements**

- Donor and charity registration, profile management, identity/tax-ID verification
- Food listing creation with type, quantity, allergens, prep time, "best before" window
- Search, filter, and geo-based discovery of available donations by charities
- Automated matching engine that proposes/auto-assigns the nearest suitable charity
- Real-time notifications (push/SMS/email) on new listings and confirmed matches
- Pickup/delivery scheduling, live status tracking, proof-of-collection (QR/e-signature)
- Admin dashboard for onboarding approval, dispute handling, content moderation
- Reporting and analytics: volume redistributed, response times, charity coverage
- Rating/feedback mechanism between donors and charities

#### **1.3 Non-Functional Requirements**

- **Scalability** — support growing donors/charities/listings across cities without redesign
- **Reliability** — matching/notification must work even under partial failures (perishable food)
- **Availability** — 24/7 accessibility, since surplus arises anytime (e.g., after events)
- **Performance** — listings matched and notified within seconds to preserve food safety windows
- **Security** — protect personal data, prevent fraudulent listings/claims
- **Maintainability** — features upgradable independently as nonprofit needs evolve
- **Usability** — simple mobile-first interfaces for restaurant staff and volunteers
- **Interoperability** — integration with maps, SMS/push gateways, logistics partners

#### **1.4 Key Quality Attributes**

Given the time-critical, geographically distributed, multi-stakeholder nature of the problem, the architecture must prioritize scalability, availability, performance, reliability, security, and maintainability — these directly shaped the architectural choice below.

## CHAPTER 2-SELECT A SUITABLE SOFTWARE ARCHITECTURE

### 2.1 Selected Style

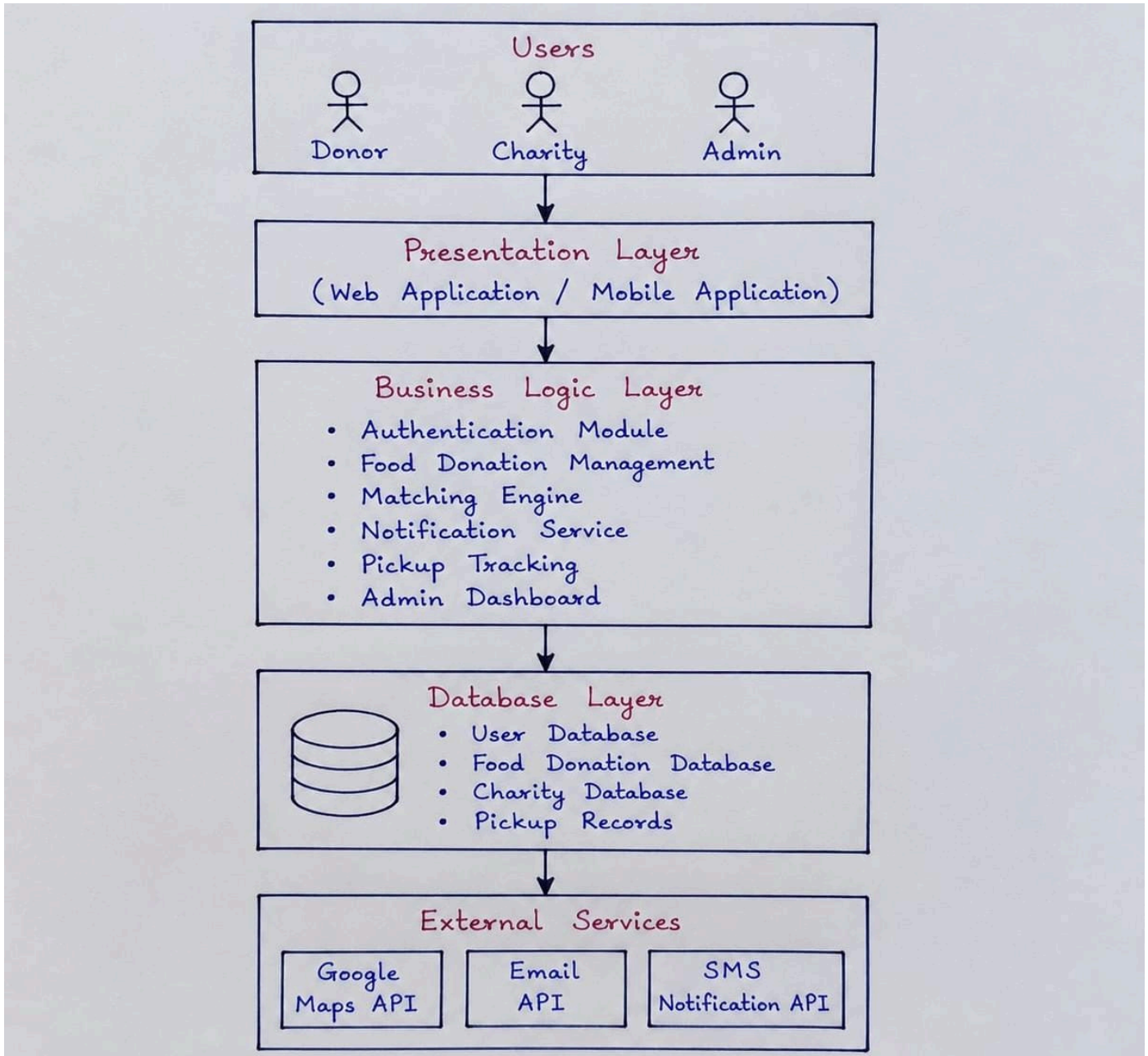
Microservices Architecture with Event-Driven Communication (a hybrid), fronted by an API Gateway. Each business capability — donor management, charity management, food listings, matching, notifications, logistics tracking, verification, analytics — is an independently deployable service. Services communicate synchronously (REST via the API Gateway) for direct user actions, and asynchronously (via an event bus) for time-sensitive workflows like matching and notification.

### 2.2 Justification

- **Independent scalability** — Matching/Notification traffic spikes after events or peak hours; these can scale independently of low-traffic services like admin verification.
- **Time-critical event handling** — The event bus lets a new listing immediately trigger matching/notification asynchronously without blocking the donor's request.
- **Fault isolation** — If Notification or an SMS gateway goes down, listing and browsing continue; events queue and retry instead of the platform failing.
- **Independent deployability** — The matching algorithm or a new logistics partner can be updated without redeploying everything — important for a small nonprofit team shipping incrementally.
- **Future growth** — New capabilities (payments, volunteer-driver logistics, multi-city expansion) can be added as new services subscribing to existing events.
- **Technology flexibility** — Geo-matching, notification dispatch, and analytics can each use the best-fit stack.

A layered/monolithic architecture was considered but rejected: it would tightly couple time-sensitive matching/notification logic to the rest of the system, block independent scaling, and risk one failure taking down the entire donation pipeline — unacceptable for perishable food.

## CHAPTER 3-SOFTWARE ARCHITECTURE DIAGRAM



**Data flow:** Food Listing Service publishes a Food Listed event → Matching Service consumes it and publishes Match Found → Notification, Logistics Tracking, and Analytics services all consume that event asynchronously.

## CHAPTER 4-EXPLAIN COMPONENTS AND THEIR INTERACTIONS

### 4.1 Component Responsibilities

| Component                        | Purpose                    | Key Responsibilities                                                         |
|----------------------------------|----------------------------|------------------------------------------------------------------------------|
| Client Apps                      | Entry points per user type | Listing creation, browsing/claiming, tracking, admin oversight               |
| API Gateway                      | Single entry point         | AuthN/AuthZ (JWT/OAuth2), routing, rate limiting, load balancing             |
| Donor Management Service         | Donor accounts             | Registration, profile, KYC/verification, donation history                    |
| Charity Management Service       | Charity accounts           | Registration, verification status, service area, capacity                    |
| Food Listing Service             | Surplus food postings      | Create/update/expire listings, images, pickup window                         |
| Matching & Allocation Service    | Core matching logic        | Consumes Food Listed, applies geo-proximity/capacity rules, proposes matches |
| Notification Service             | Real-time alerts           | Push/SMS/email to charities on match, to donors on confirmation              |
| Pickup/Delivery Tracking Service | Fulfillment                | Scheduling, live status, proof-of-collection, delivery partner integration   |
| Verification & Admin Service     | Governance                 | Approves orgs, handles disputes/fraud, moderates content                     |
| Analytics & Reporting Service    | Impact measurement         | Aggregates events into dashboards (meals saved, response time, coverage)     |
| Event Bus (Kafka/RabbitMQ)       | Async backbone             | Decouples producers/consumers, guarantees delivery of critical events        |
| Databases (per service)          | Persistence                | Each service owns its schema; plus Object Storage for images                 |

### 4.2 Communication Mechanisms

- Client ↔ Gateway: synchronous REST/HTTPS
- Gateway ↔ Services: routed, authenticated requests
- Service ↔ Service (time-critical): asynchronous events via the Event Bus
- Service ↔ Database: each service owns its own database exclusively
- Service ↔ External Systems: outbound REST calls to Maps, SMS/Push providers, logistics partners

### 4.3 Overall Workflow

1. Donor creates a listing → Food Listing Service stores it and publishes Food Listed
2. Matching Service consumes the event, applies proximity/capacity logic, publishes Match Found
3. Notification Service alerts the charity; on confirmation, alerts the donor
4. Pickup/Delivery Tracking Service manages handover and status
5. Analytics Service consumes events throughout to update impact dashboards

## CHAPTER 5-DISCUSS QUALITY ATTRIBUTES

| Attribute       | How It's Addressed                                 | Mechanisms                                                  |
|-----------------|----------------------------------------------------|-------------------------------------------------------------|
| Scalability     | Services scale independently based on demand       | Containerized services, auto-scaling, load-balanced gateway |
| Security        | Centralized auth, least-privilege data access      | OAuth2/JWT, DB isolation per service, TLS, RBAC for admin   |
| Performance     | Time-critical work decoupled from user requests    | Async event bus, geo-indexed queries, caching               |
| Reliability     | Failures don't cascade; messages retried, not lost | Durable queues, retry/dead-letter queues, circuit breakers  |
| Maintainability | Small, single-responsibility services              | Clear boundaries, versioned APIs, independent CI/CD         |
| Availability    | No single outage takes down the platform           | Multiple instances, DB replication, persistent event bus    |

## CHAPTER 6-JUSTIFY ARCHITECTURAL DECISIONS

### 6.1 Rationale

Microservices give structural modularity for independently evolvable stakeholder interfaces; the event-driven backbone handles food's short usable lifespan by making matching/alerting low-latency and asynchronous rather than blocking.

### 6.2 Trade-offs

- **Complexity vs. resilience** — More operational overhead than a monolith, but justified by reliability/scaling needs
- **Eventual consistency vs. responsiveness** — Small delay in charity dashboards, traded for overall system decoupling
- **Operational cost vs. nonprofit budget** — Mitigate via managed cloud services or starting as a modular monolith that splits later
- **Data duplication vs. service autonomy** — Some duplicated data (e.g., cached charity name) traded for services not blocking on each other's schema changes

### 6.3 Future Enhancements, Integration, and Long-Term Maintainability

- New features (payments, volunteer logistics, rewards) added as new services subscribing to existing events
- City/country expansion via additional service instances and regional routing
- Integration with government reporting systems or other NGOs via existing Gateway/Event Bus integration points
- Narrow service responsibilities make it easier for rotating volunteer developers to safely contribute



